

FILE & SYSTEM STRUCTURE

Physical Storage Media

1

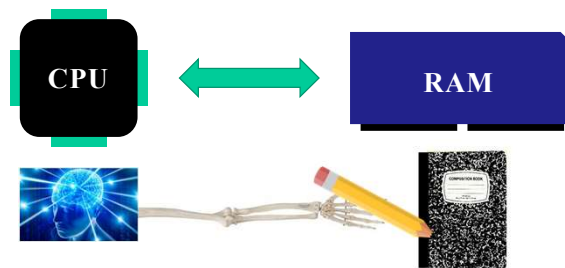
2025/8/11

Lecture Outline

- **Memory & Caching**
 - **How Memory Looks** 
 - How Memory Is Used
- Indexed Files
- B+ Trees

So... What is a Computer?

- At the simplest level, think of a computer as being two components:
 - CPU: Central Processing Unit (The “brain”. When any operation is run, it’s running in the CPU. Takes in inputs and evaluates an output.)
 - RAM: Random Access Memory (The “notebook”. Where data is kept track of, and stored between operations. Inputs are read from here and outputs are written here.)



RAM (Random-Access Memory)

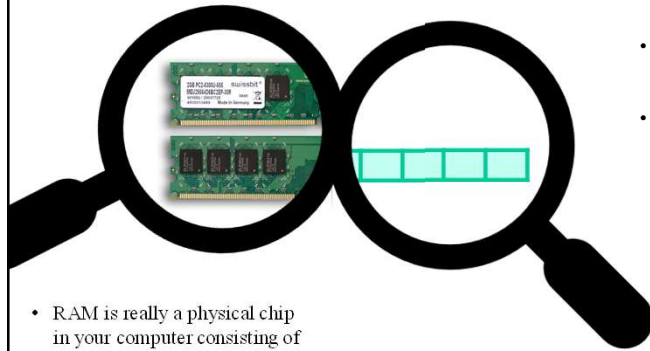
- RAM is where the programs you run store their data.**
 - Data structures, variables, method call frames, etc. all stored here!
- Often just called “Memory” or “Main Memory”



Process Name	Memory	Compressed M...	Threads	
kernel_task	1.18 GB	0 bytes	144	
IntelIJ IDEA	1,018.0 MB	194.7 MB	66	
Microsoft PowerPoint	845.1 MB	238.9 MB	18	
WindowServer	330.7 MB	170.9 MB	8	
msiexec.exe	320.8 MB	239.4 MB	3	
Mattermost Helper	315.4 MB	32.0 MB	19	
Google Chrome	291.7 MB	17.5 MB	31	
Google Chrome Helper (Rend...	243.4 MB	91.5 MB	14	
zoom.us	239.7 MB	61.8 MB	20	
Google Chrome Helper (Rend...	238.6 MB	26.7 MB	14	
Google Chrome Helper (GPU)	235.2 MB	19.7 MB	10	
Google Chrome Helper (Rend...	203.4 MB	27.9 MB	16	
Sublime Text	189.5 MB	170.9 MB	12	
spindump	168.4 MB	80.0 MB	3	
SystemUIServer	148.5 MB	24.9 MB	4	
Finder	139.9 MB	56.9 MB	4	
java	128.2 MB	61.3 MB	24	
java	126.3 MB	110.3 MB	23	
java	124.4 MB	27.8 MB	28	
mds_Mores	115.5 MB	36.2 MB	4	
Mattermost	112.3 MB	37.5 MB	44	
Color Tuner Blocker	109.1 MB	49.2 MB	9	
Google Chrome Helper (Rend...	102.8 MB	33.0 MB	16	
Mail	91.4 MB	25.6 MB	7	
Google Chrome Helper (Rend...	90.1 MB	62.4 MB	13	
Google Chrome Helper (Rend...	88.1 MB	54.8 MB	13	
Mattermost Helper	82.5 MB	44.8 MB	5	
Google Chrome Helper (Rend...	77.4 MB	32.5 MB	13	
Google Chrome Helper (Rend...	72.7 MB	51.4 MB	13	

MEMORY PRESSURE		Physical Memory: 16.00 GB
		Memory Used: 9.81 GB
		Cached Files: 1.94 GB
		Sleep Used: 628.0 MB

Think of RAM as a Giant Array!



- RAM is really a physical chip in your computer consisting of complicated circuitry

- Fortunately, as programmers we don't need to understand the circuitry below!
- We think about RAM through the **abstraction** of a giant array:
 - Stores data in specific locations
 - Indices to describe those locations (we call them addresses for memory)
 - We can jump to any index ("random access")

LOW-LEVEL REALITY HIGH-LEVEL ABSTRACTION

Simple Data in RAM

```
int a = 5;
char letter = 'z'
```

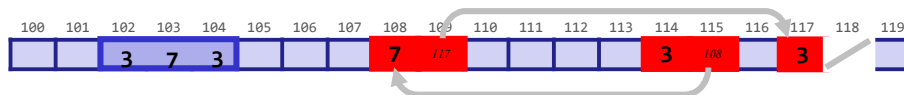
a: refers to address 107
letter: refers to address 113

100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119
							5						'z'						

Data Structures in RAM

```
int[] array = new int[3];
array[0] = 3;
array[1] = 7;
array[2] = 3;
```

```
Node front = new Node(3);
front.next = new Node(7);
front.next.next = new Node(3);
```



- An array is a contiguous block of memory (a bunch of slots next to each other)
- A linked list is a series of nodes, with references to each other
 - How to reference? Simply store the address!
 - Nodes do not need to be contiguous, or even in order

Lecture Outline

- **Memory & Caching**
 - How Memory Looks
 - **How Memory Is Used** ◀
- **Indexed Files**
- B+ Trees

Buying Bubble Tea

- Suppose there's an essential grocery you need every few hours
- As soon as you realize you're thirsty, you:
 - (1) Walk to the store (2) Buy a bubble tea (3) Walk back home (4) Enjoy
- But you repeat this multiple times a day! It takes so long to walk to the store, and that's a lot of time spent away from the lecture...



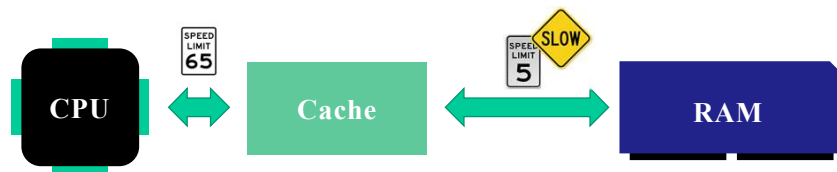
Buying Bubble Tea: Planning Ahead

- Could this be more efficient?
- Since you know it's likely you'll want another bubble tea in a few hours, what if you do what any reasonable person would: buy a bubble tea minifridge and store a handful closer to home!



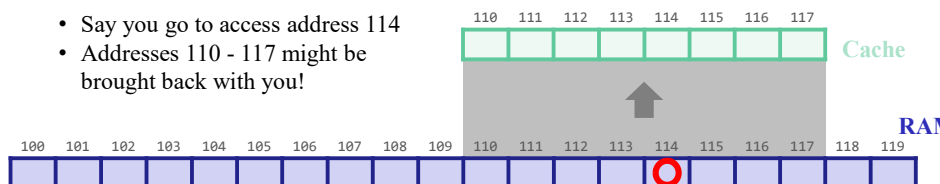
Cache

- Pronounced “cash”
- Intermediate storage between the CPU and RAM
 - RAM takes a long time to access, but is gigantic. Cache is much faster (closer to the CPU where data gets processed), but smaller.
- Store a copy of some data here
 - When we’re about to go grab an address from RAM, we check the cache first – and we *love* when the data’s there, because it’s much faster!



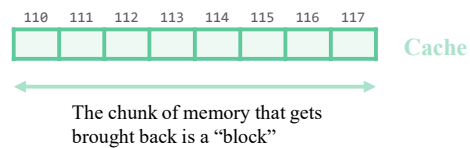
Bringing More Data Back

- If we need to go all the way to RAM, might as well make it count!
- Your computer *automatically* grabs a whole chunk of data around each address from RAM when you access it
 - That chunk of data is then copied to the cache
 - Your computer knows it’s likely you’ll want a nearby address soon
 - Bringing back multiple addresses of data costs nothing: the hardware is designed to grab many at a time
- Say you go to access address 114
- Addresses 110 - 117 might be brought back with you!



Cache Implications: Arrays

- This has a major impact on programming with arrays!
 - Suppose we're looping through everything in an array. When we access index 0, we grab a whole chunk of the array and put it in the cache – now the next (block size) accesses are much faster!
 - For a short array, we might even grab the whole thing and bring it into the cache

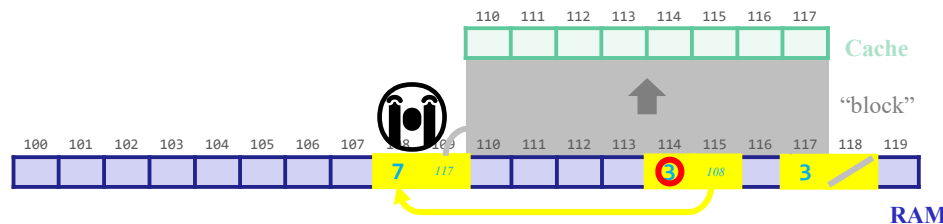


Characterizing Cache-Friendly Programs

- **Spatial locality:** tendency for programs to access locations nearby to recent locations
 - Plenty of our programs exhibit spatial locality: e.g. looping through an array
- **Temporal locality:** tendency for programs to access data that was recently accessed
 - Plenty of our programs exhibit temporal locality: e.g. adding to sum variable over and over
- Programs with spatial and temporal locality benefit the most from caching!

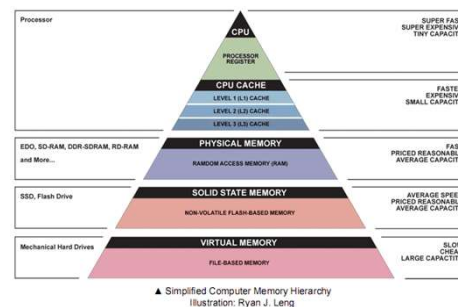
Cache Implications: Linked Lists

- Linked lists can be spread out all over the RAM array
 - Do not exhibit strong spatial locality!
- Don't get the same cache benefits – frequently the next list node is far enough away that it's not included on the same block



Memory Architecture

- Typically multiple caches (progressively smaller and faster: L1, L2, & L3)
- Beyond RAM is the disk, which is way, way, WAY slower – but *much* bigger, & disk memory persists when the computer is off (RAM gets cleared)
 - **Similar idea: chunk of data gets pulled into RAM when accessed on disk (called a “page”)**



Asymptotic Analysis, Meet The Real World

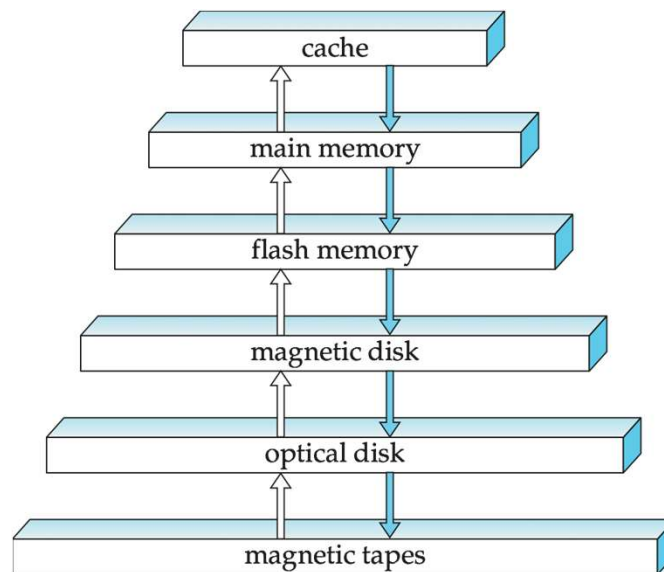
- Asymptotic analysis tells us iterating through an array and a linked list are the same complexity class (linear)
 - This is still true: *growth rates* are the same, and asymptotic analysis is a helpful tool to capture that
 - But arrays are frequently a *significant* constant factor faster due to cache performance! One area asymptotic analysis isn't a good tool for

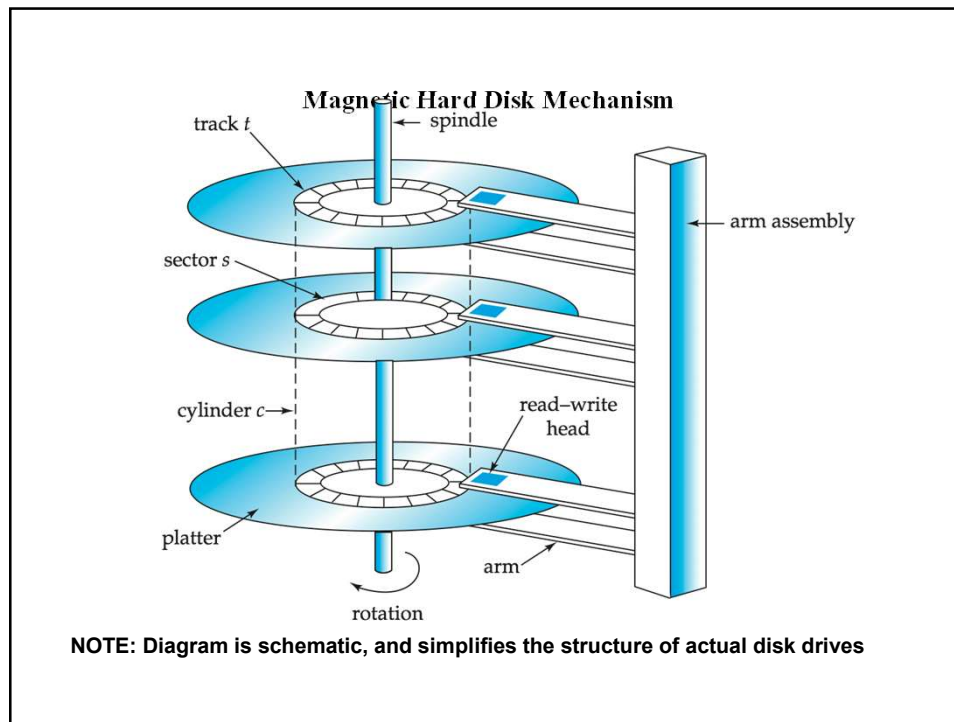
“Latency Numbers Everyone Should Know” from [Jeff Dean](#), Senior Fellow at Google

L1 cache reference	0.5 ns	
Branch mispredict	5 ns	
L2 cache reference	7 ns	
Mutex lock/unlock	100 ns	
Main memory reference	100 ns	
Compress 1K bytes with Zippy	10,000 ns	0.01 ms
Send 1K bytes over 1 Gbps network	10,000 ns	0.01 ms
Read 1 MB sequentially from memory	250,000 ns	0.25 ms
Round trip within same datacenter	500,000 ns	0.5 ms
Disk seek	10,000,000 ns	10 ms
Read 1 MB sequentially from network	10,000,000 ns	10 ms
Read 1 MB sequentially from disk	30,000,000 ns	30 ms
Send packet CA→Netherlands→CA	150,000,000 ns	150 ms

Where
 1 ns = 10^{-9} seconds
 1 ms = 10^{-3} seconds

Physical Storage Hierarchy





Magnetic Disks

- **Read-write head**
 - Positioned very close to the platter surface (almost touching it)
 - Reads or writes magnetically encoded information.
- Surface of platter divided into circular **tracks**
 - Over 16,000 tracks per platter on typical hard disks
- Each track is divided into **sectors**.
 - A sector is the smallest unit of data that can be read or written.
 - Sector size typically 512 bytes
 - Typical sectors per track: 200 (on inner tracks) to 400 (on outer tracks)
- To read/write a sector
 - disk arm swings to position head on right track
 - platter spins continually; data is read/written as sector passes under head
- Head-disk assemblies
 - multiple disk platters on a single spindle (typically 2 to 4)
 - one head per platter, mounted on a common arm.
- **Cylinder i** consists of i^{th} track of all the platters

Magnetic Disks (Cont.)

- Earlier generation disks were susceptible to head-crashes
 - Surface of earlier generation disks had metal-oxide coatings which would disintegrate on head crash and damage all data on disk
 - Current generation disks are less susceptible to such disastrous failures, although individual sectors may get corrupted
- **Disk controller** – interfaces between the computer system and the disk drive hardware.
 - accepts high-level commands to read or write a sector
 - initiates actions such as moving the disk arm to the right track and actually reading or writing the data
 - Computes and attaches **checksums** to each sector to verify that data is read back correctly
 - If data is corrupted, with very high probability stored checksum won't match recomputed checksum
 - Ensures successful writing by reading back sector after writing it
 - Performs remapping of bad sectors

21

2025/8/11

Optimization of Disk-Block Access

- **Block** – a contiguous sequence of sectors from a single track
 - data is transferred between disk and main memory in blocks
 - sizes range from 512 bytes to several kilobytes
 - Smaller blocks: more transfers from disk
 - Larger blocks: more space wasted due to partially filled blocks
 - Typical block sizes today range from 4 to 16 kilobytes
- **Disk-arm-scheduling** algorithms order pending accesses to tracks so that disk arm movement is minimized
 - **elevator algorithm** : move disk arm in one direction (from outer to inner tracks or vice versa), processing next request in that direction, till no more requests in that direction, then reverse direction and repeat

22

2025/8/11

Optimization of Disk Block Access (Cont.)

- **File organization** – optimize block access time by organizing the blocks to correspond to how data will be accessed
 - E.g. Store related information on the same or nearby cylinders.
 - Files may get **fragmented** over time
 - E.g. if data is inserted to/deleted from the file
 - Or free blocks on disk are scattered, and newly created file has its blocks scattered over the disk
 - Sequential access to a fragmented file results in increased disk arm movement
 - Some systems have utilities to **defragment** the file system, in order to speed up file access

TERMS

- Locality of References:
 - When a record is read from disk, the next request is likely to come from near the same place in the file.
- Cluster
 - Smallest unit of the allocation, usually several sectors.
- Extend
 - A group of physically contiguous clusters.
- Internal fragmentation
 - Wasted space within sector if record size does not match sector size; wasted space within cluster if the size is not a multiple of cluster size.

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....00000

- A byte is eight contiguous bits:

0101111110011010101010110000000000.....00000

25

2025/8/11

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....00000

- A byte is eight contiguous bits:

01011111100110101010110000000000.....00000

26

2025/8/11

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....00000

- A byte is eight contiguous bits:

01011111100110101010110000000000.....00000

27

2025/8/11

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....00000

- A byte is eight contiguous bits:

01011111100110101010110000000000.....00000

- 1024 bytes is one Kilobyte (KB), e.g., your textbook.

28

2025/8/11

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....00000

- A byte is eight contiguous bits:

01011111100110101010110000000000.....00000

- 1024 bytes is one Kilobyte (KB).
- 1024 KB is one Megabyte (MB), a high resolution photograph.

29

2025/8/11

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....00000

- A byte is eight contiguous bits:

01011111100110101010110000000000.....00000

- 1024 bytes is one Kilobyte.
- 1024 KB is one Megabyte (MB).
- 1024 MB is one Gigabyte (GB), e.g., a DVD quality movie.

30

2025/8/11

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....000000

- A byte is eight contiguous bits:

01011111100110101010110000000000.....000000

- 1024 bytes is one Kilobyte.
- 1024 KB is one Megabyte.
- 1024 MB is one Gigabyte.
- 1024 GB is one Terabyte (TB), all text in the library of congress.

31

2025/8/11

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....000000

- A byte is eight contiguous bits:

01011111100110101010110000000000.....000000

- 1024 bytes is one Kilobyte.
- 1024 KB is one Megabyte.
- 1024 MB is one Gigabyte.
- 1024 GB is one Terabyte (TB).
- 1024 TB is one Petabyte (PB), entire multimedia collection at LoC.

32

2025/8/11

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....000000

- A byte is eight contiguous bits:

01011111100110101010110000000000.....000000

- 1024 bytes is one Kilobyte.
- 1024 KB is one Megabyte.
- 1024 MB is one Gigabyte.
- 1024 GB is one Terabyte.
- 1024 TB is one Petabyte (PB).
- 1024 PB is one Exabyte (XB), record all phone conversations in a year.

33

2025/8/11

TERMINOLOGY

- Computers represent data as a sequence of zero and ones, termed bits:

0101111110011010101010110000000000.....000000

- A byte is eight contiguous bits:

01011111100110101010110000000000.....000000

- 1024 bytes is one Kilobyte.
- 1024 KB is one Megabyte.
- 1024 MB is one Gigabyte.
- 1024 GB is one Terabyte.
- 1024 TB is one Petabyte.
- 1024 PB is one Exabyte.
- 1024 XB is one Zetabyte (ZB), all uncompressed medical data.

34

2025/8/11

FILE & SYSTEM STRUCTURE (Cont...)

A database system is organized as several layers of software:

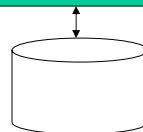
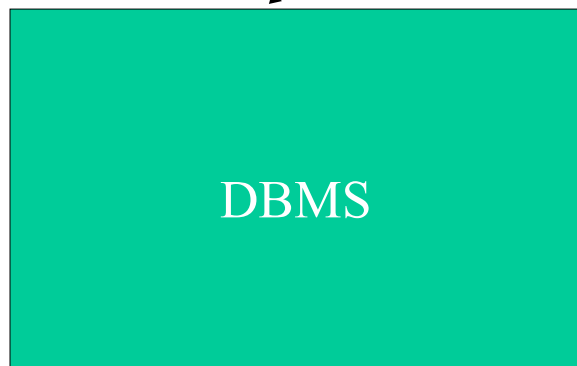
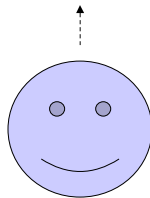
- Query parser: translates a higher level query language to an internal representation
- Query optimizer: transforms the internal representation to an efficient execution paradigm
- Concurrency control and crash recovery: ensures consistency of data in the presence of multiple concurrent update operations and crash-recoveries.
- Index methods: efficient retrieval of records for fast retrieval and update operations
- Abstraction of multiple records on a disk page: implements the concept of multiple records on a disk page.

35

2025/8/11

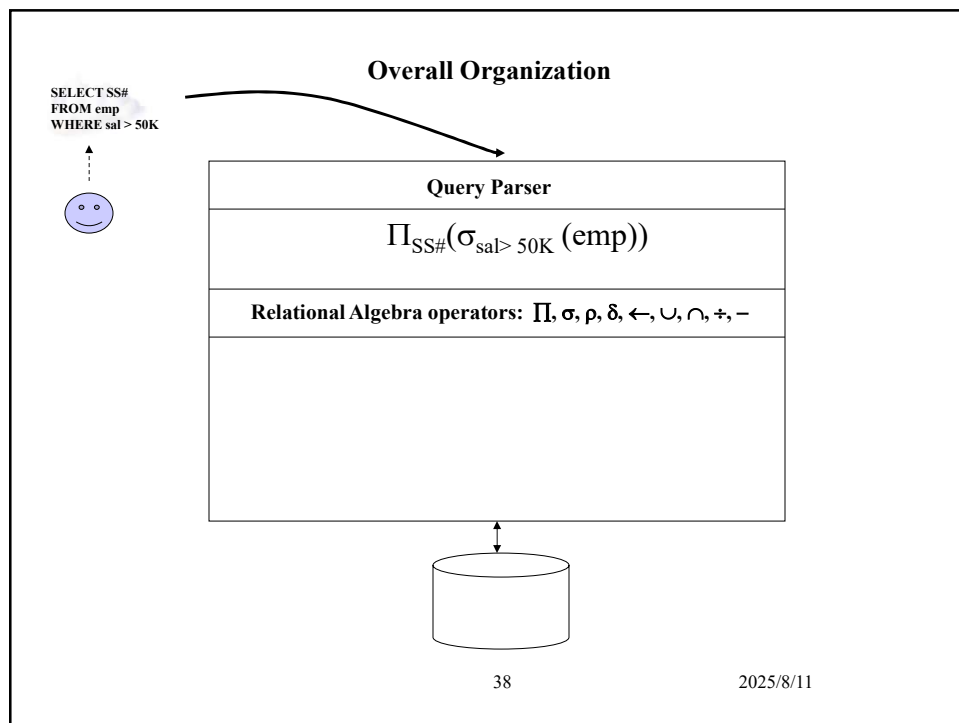
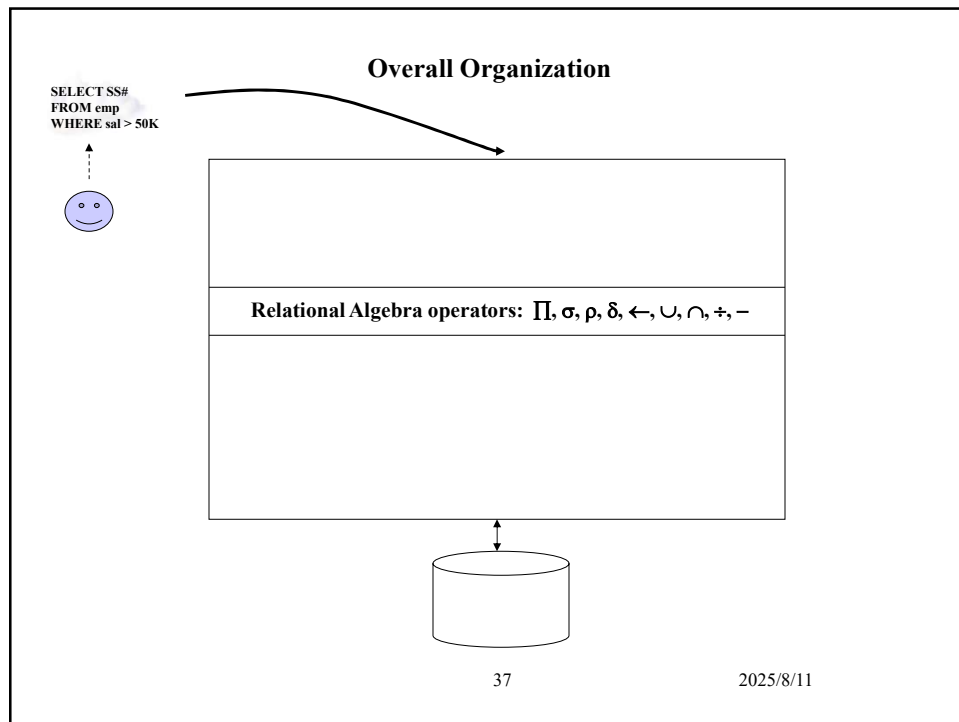
BIG PICTURE

```
SELECT SS#  
FROM emp  
WHERE sal > 50K
```



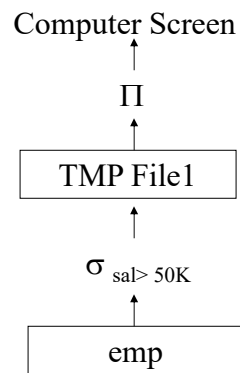
36

2025/8/11



QUERY TREE

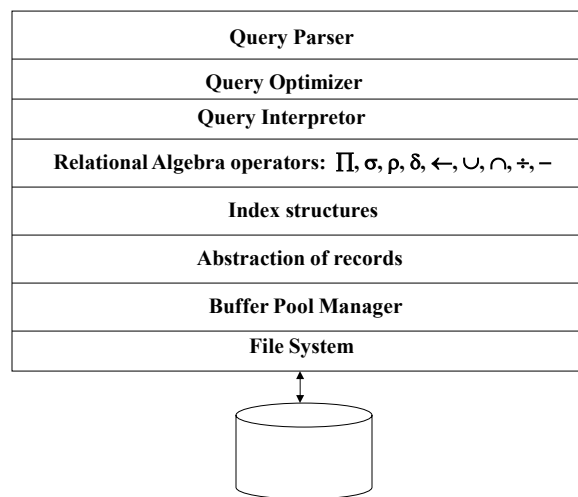
- $\Pi_{SS\#}(\sigma_{sal > 50K}(emp))$ becomes a query tree:



39

2025/8/11

Overall Organization



40

2025/8/11

FILE & SYSTEM STRUCTURE (Cont...)

- Buffer manager maintains a portion of memory that is conceptualized as disk page frames. It maintains which disk pages are memory resident. It also implements a replacement policy in order to swap a page out in favor of another disk page that is being referenced. This happens because the number of memory page frames is significantly smaller than the number of disk pages.
- File manager provides the following services: create a file, delete a file, read a disk page into a specific memory address given the physical address of disk page on the secondary storage device, write a disk page from a memory address on to the appropriate physical disk address, insert a page into a file, modify a page, and delete a page from a file.

41

2025/8/11

FILE & SYSTEM STRUCTURE (Cont...)

- When a program requests a disk page (by specifying its address), the buffer manager takes the following steps:
 1. Check if the page is in the buffer.
 2. If it is then pass its address to the calling program.
 3. Otherwise, read the page from the disk into the buffer, possibly replacing some other page, and then pass its address to the calling program.
- *Pinned blocks*: Occasionally, the DBMS needs to specifically indicate that some blocks have to be kept in the buffer until released by unpinning them. These blocks are termed *pinned*.
- *Forced writing of blocks to disks*: To preserve the consistency of the database during crash-recovery, the DBMS might force the buffer manager to flush some blocks to disks.

42

2025/8/11

Lecture Outline

- Memory & Caching
 - How Memory Looks
 - How Memory Is Used
- **Indexed Files** 
- B+ Trees

Index Evaluation Metrics

- Access types supported efficiently. E.g.,
 - Records with a specified value in the attribute
 - Records with an attribute value falling in a specified range of values.
- Access time
- Insertion time
- Deletion time
- Space overhead

Ordered Indices

- In an **ordered index**, index entries are stored sorted on the search key value.
- **Clustering index**: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
 - Also called **primary index**
 - The search key of a primary index is usually but not necessarily the primary key.
- **Secondary index**: an index whose search key specifies an order different from the sequential order of the file. Also called **nonclustering index**.
- **Index-sequential file**: sequential file ordered on a search key, with a clustering index on the search key.

Dense Index Files

- **Dense index** — Index record appears for every search-key value in the file.
- E.g. index on *ID* attribute of *instructor* relation

10101	→	10101	Srinivasan	Comp. Sci.	65000	→
12121	→	12121	Wu	Finance	90000	→
15151	→	15151	Mozart	Music	40000	→
22222	→	22222	Einstein	Physics	95000	→
32343	→	32343	El Said	History	60000	→
33456	→	33456	Gold	Physics	87000	→
45565	→	45565	Katz	Comp. Sci.	75000	→
58583	→	58583	Califieri	History	62000	→
76543	→	76543	Singh	Finance	80000	→
76766	→	76766	Crick	Biology	72000	→
83821	→	83821	Brandt	Comp. Sci.	92000	→
98345	→	98345	Kim	Elec. Eng.	80000	→

Dense Index Files (Cont.)

- Dense index on *dept_name*, with *instructor* file sorted on *dept_name*

Biology		76766	Crick	Biology	72000	
Comp. Sci.		10101	Srinivasan	Comp. Sci.	65000	
Elec. Eng.		45565	Katz	Comp. Sci.	75000	
Finance		83821	Brandt	Comp. Sci.	92000	
History		98345	Kim	Elec. Eng.	80000	
Music		12121	Wu	Finance	90000	
Physics		76543	Singh	Finance	80000	
		32343	El Said	History	60000	
		58583	Califieri	History	62000	
		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		33465	Gold	Physics	87000	

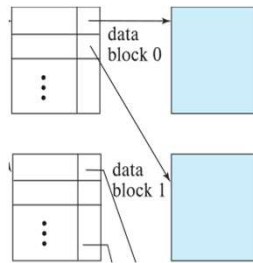
Sparse Index Files

- Sparse Index:** contains index records for only some search-key values.
 - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value K we:
 - Find index record with largest search-key value $< K$
 - Search file sequentially starting at the record to which the index record points

10101		10101	Srinivasan	Comp. Sci.	65000	
32343		12121	Wu	Finance	90000	
76766		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		32343	El Said	History	60000	
		33456	Gold	Physics	87000	
		45565	Katz	Comp. Sci.	75000	
		58583	Califieri	History	62000	
		76543	Singh	Finance	80000	
		76766	Crick	Biology	72000	
		83821	Brandt	Comp. Sci.	92000	
		98345	Kim	Elec. Eng.	80000	

Sparse Index Files (Cont.)

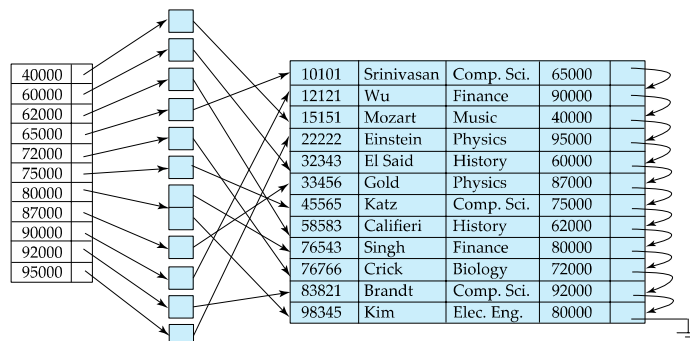
- Compared to dense indices:
 - Less space and less maintenance overhead for insertions and deletions.
 - Generally slower than dense index for locating records.
- Good tradeoff:**
 - for clustered index: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.



- For unclustered index: sparse index on top of dense index (multilevel index)

Secondary Indices Example

- Secondary index on salary field of instructor



- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary indices have to be dense

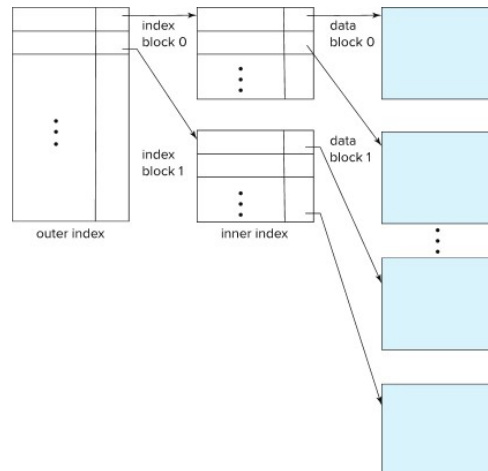
Clustering vs Nonclustering Indices

- Indices offer substantial benefits when searching for records.
- BUT: indices imposes overhead on database modification
 - when a record is inserted or deleted, every index on the relation must be updated
 - When a record is updated, any index on an updated attribute must be updated
- Sequential scan using clustering index is efficient, but a sequential scan using a secondary (nonclustering) index is expensive on magnetic disk
 - Each record access may fetch a new block from disk
 - Each block fetch on magnetic disk requires about 5 to 10 milliseconds

Multilevel Index

- If index does not fit in memory, access becomes expensive.
- Solution: treat index kept on disk as a sequential file and construct a sparse index on it.
 - outer index – a sparse index of the basic index
 - inner index – the basic index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.

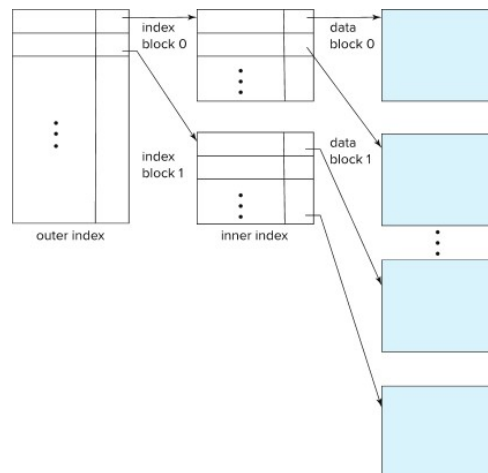
Multilevel Index (Cont.)



Multilevel Index

- If index does not fit in memory, access becomes expensive.
- Solution: treat index kept on disk as a sequential file and construct a sparse index on it.
 - outer index – a sparse index of the basic index
 - inner index – the basic index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.

Multilevel Index (Cont.)



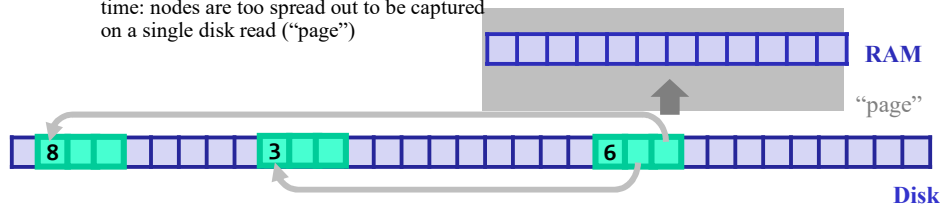
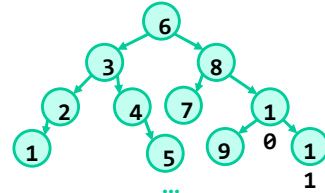
Lecture Outline

- Memory & Caching
 - How Memory Looks
 - How Memory Is Used
- Indexed Files
- B+ Trees ◀

Minimizing Disk Accesses

- Let's consider a truly massive amount of data
 - too much data to fit in RAM (some has to be stored on disk)
 - This is very common! For example, a database
- What will happen if we store it in a giant AVL tree? Say height 40, so $2^{40} = 1.1 * 10^{12}$ nodes
 - Similar problem as before, just with disk this time: nodes are too spread out to be captured on a single disk read ("page")

A laptop these days might have
8 GB of RAM
250 GB of Disk space

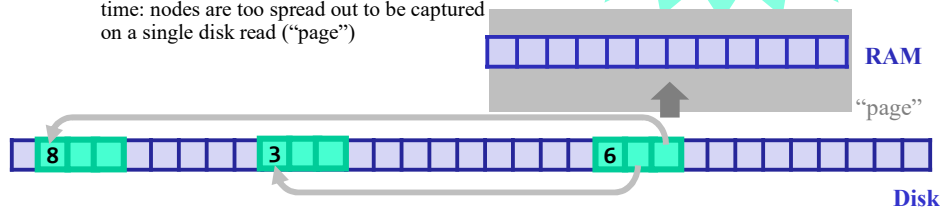


Minimizing Disk Accesses

- Let's consider a truly massive amount of data
 - too much data to fit in RAM (some has to be stored on disk)
 - This is very common! For example, a database
- What will happen if we store it in a giant AVL tree? Say height 40, so $2^{40} = 1.1 * 10^{12}$ nodes
 - Similar problem as before, just with disk this time: nodes are too spread out to be captured on a single disk read ("page")

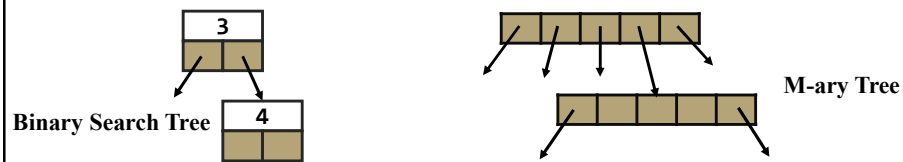
A laptop these days might have
8 GB of RAM
250 GB of Disk space

Our goal:
A data structure
optimized to make as few
disk accesses as
possible! (suitable for
large amounts of data)



Minimizing Disk Accesses: Idea 1/3

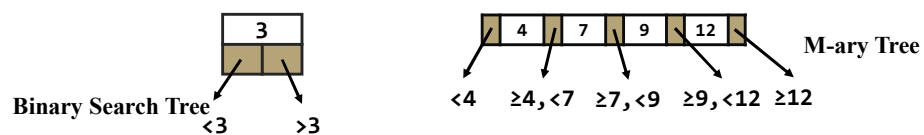
- Idea: Node size of our BSTs/AVL Trees is small, but we move a whole page at a time in from disk
 - What if we could stuff more useful information in each node?
- First, let's generalize the number of children: while a Binary Tree has at most 2 children, an "M-ary" Tree has at most M children



- This is incomplete: How do we keep these children organized? What happens to the key?

Minimizing Disk Accesses: Idea 2/3

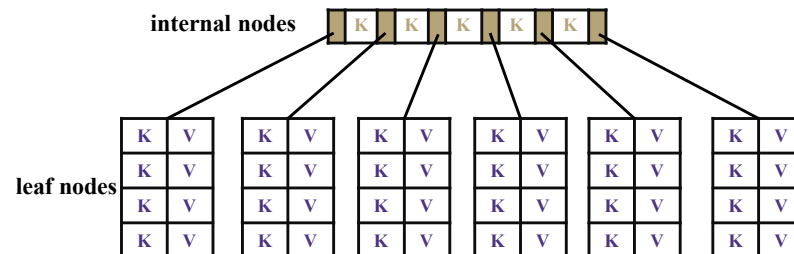
- How do we keep these children organized? What happens to the key?
- In a Binary Search Tree, the **key** divides the contents of the **child subtrees**
 - Same principle: in our tree, we have a **sorted** array of M-1 **keys**, which divide the contents of **child subtrees**



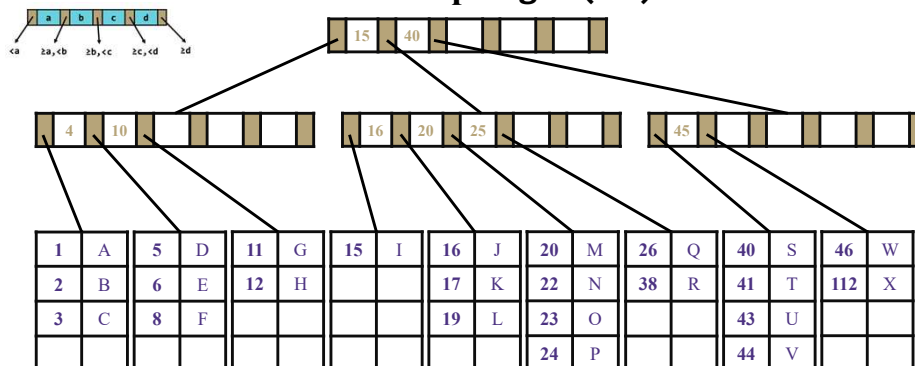
- Suppose we want to store values too (implement the Map ADT, useful for a database)? Where should we put those?

Minimizing Disk Accesses: Idea 3/3

- We can pack all the key/value pairs into the leaf nodes, to really maximize stuffing in useful information
- This is a **B+ Tree**: a disk-friendly data structure™
 - Internal nodes become “fenceposts” that guide us to the leaves, leaves have all the data
 - Both types of nodes sized to fit on a single page!



B+ Tree Example: get(23)



Why Are B+ Trees so Disk-Friendly? (Summary)

1. We **minimized the height** of the tree by adding more keys/potential children at every node. Because the nodes are more spread out at a shallower place in the tree, it takes fewer nodes (disk-accesses) to traverse to a leaf.
2. All relevant information about a single node **fits in one page** (If it's an internal node: all the keys it needs to determine which branch it should go down next. If it's a leaf: the relevant K/V pairs).
3. We use **as much of the page as we can**: each node contains many keys that are all brought in at once with a single disk access, basically "for free".
4. The time needed to do a search within a node is **insignificant** compared to disk access time, so looking within a node is also "free".

What About Inserting/Removing?

- Takeaways:
 - Disk lookups are slow, so if you have large amounts of data (enough that it spills over onto the disk), consider using a B+ trees!
 - Databases use these *all* the time! Even the very core file system in your computer makes use of B+ trees
 - B+ trees minimize the # of disk accesses by stuff as much data into each node so that the height of tree is short, and every node requires just one disk access